Bugs Report:

1. Underflow signal implementation issue

- underflow is implemented as a combinational signal instead of a sequential one.
- Modification : made it following clock edge in the read always block

2. Missing condition

- If both read and write enables are high simultaneously:
  - When the FIFO is empty, only writing will take place.
  - When the FIFO is full, only reading will take place.

3. Reset Behavior error

- Bug: there are some signals which their output haven't been specified when reset signal takes place

```
vsim work.FIFO_top -coverage -voptargs="+acc=mnpr"

# Add everything safely
add wave /FIFO_top/fifo_vif/*
add wave -r /FIFO_top/dut/*

run -all
```

Label	Design Requirement Description	Stimulus Generation	Functional Coverage	Functionality Check
FIFO_1	When the reset is asserted, the output ant pointer value should be low	Directed at the start of the simulation	-	Assertions + Scoreboard compares expected vs actual stored data.
FIFO_2	When wr_en is asserted and FIFO is not full, data_in shall be written into memory and wr_ack shall assert.	Constrained random writes.	Cover cross coverage between wr_en, rd_en wr_ack, and write success cases.	Assertions + Scoreboard compares expected vs actual stored data..
FIFO_3	When wr_en is asserted while FIFO is full, no write shall occur and overflow shall assert.	Directed + random (force full condition then write).	Cover cross coverage between wr_en, rd_en, overflow	Assertions + Scoreboard compares expected vs actual stored data..
FIFO_4	When rd_en is asserted and FIFO is not empty, data shall be read correctly from memory.	Constrained random reads.	Cover cross coverage between wr_en, rd_en full	Assertions + Scoreboard compares expected vs actual stored data..
FIFO_5	When rd_en is asserted while FIFO is empty, no read shall occur and underflow shall assert.	Directed + random (force empty condition then read).	Cover cross coverage between wr_en, rd_en, underflow	Assertions + Scoreboard compares expected vs actual stored data..
FIFO_6	FIFO shall assert full when count equals FIFO depth.	Random fill sequences.	Cover full condition.	Assertions + Scoreboard compares expected vs actual stored data..
FIFO_7	Simultaneous read and write operations shall update pointers and count correctly.	Constrained random with wr_en & rd_en both high.	Cross coverage of wr_en and rd_en.	Assertions + Scoreboard compares expected vs actual stored data..
FIFO_8	Write pointer shall wrap around correctly at FIFO depth.	Directed wrap-around sequences.	Cover pointer boundary conditions.	Assertions + Scoreboard compares expected vs actual stored data..
FIFO_9	Read pointer shall wrap around correctly at FIFO depth.	Directed wrap-around sequences.	Cover pointer boundary conditions.	Assertions + Scoreboard compares expected vs actual stored data..
FIFO_10	FIFO count shall never exceed depth or go below zero.	Directed wrap-around sequences.	Cover pointer boundary conditions.	Assertions + Scoreboard compares expected vs actual stored data..

FIFO_top.sv

```
import shared_pkg::*;

module FIFO_top();

    logic clk;

    initial begin
        clk = 0;
        forever #5 clk = ~clk;
    end

    FIFO_if fifo_vif(clk);

    FIFO dut(fifo_vif);
    FIFO_monitor monitor(fifo_vif);
    FIFO_tb test(fifo_vif);


endmodule
```

```
interface FIFO_if(input logic clk);


    parameter FIFO_WIDTH = 16;
    parameter FIFO_DEPTH = 8;
    logic [FIFO_WIDTH-1:0] data_in;
    logic rst_n, wr_en, rd_en;
    logic [FIFO_WIDTH-1:0] data_out;
    logic wr_ack, overflow;
    logic full, empty, almostfull, almostempty, underflow;

    event drv_done;

    modport DUT(input data_in, wr_en, rd_en, clk, rst_n,
                output full, empty, almostfull, almostempty, wr_ack, overflow, underflow, data_out);

    modport MONITOR(input data_in, wr_en, rd_en, clk, rst_n,
                    full, empty, almostfull, almostempty, wr_ack, overflow, underflow, data_out,
                    ref drv_done);

    modport TEST(output data_in, wr_en, rd_en, rst_n,
                 input clk, full, empty, almostfull, almostempty, wr_ack, overflow, underflow, data_out,
                 ref drv_done);

endinterface
```

FIFO.sv

```
////////////////////////////////////////////////////////////////////////////////
// Author: Mohamed Ayman
// Course: Digital Verification using SV & UVM
//
// Description: FIFO Design
//
////////////////////////////////////////////////////////////////////////////////
module FIFO(FIFO_if.DUT fifo_vif);

localparam max_fifo_addr = $clog2(fifo_vif.FIFO_DEPTH);

reg [fifo_vif.FIFO_WIDTH-1:0] mem [fifo_vif.FIFO_DEPTH-1:0];

reg [max_fifo_addr-1:0] wr_ptr, rd_ptr;
reg [max_fifo_addr:0] count;

always @(posedge fifo_vif.clk or negedge fifo_vif.rst_n) begin
    if (!fifo_vif.rst_n) begin
        wr_ptr <= 0;
        fifo_vif.wr_ack <= 0; // this was added
        fifo_vif.overflow <= 0; // this was added
    end
    else if (fifo_vif.wr_en && count < fifo_vif.FIFO_DEPTH) begin
        mem[wr_ptr] <= fifo_vif.data_in;
        fifo_vif.wr_ack <= 1;
        wr_ptr <= wr_ptr + 1;
    end
    else begin
        fifo_vif.wr_ack <= 0;
        if (fifo_vif.full & fifo_vif.wr_en)
            fifo_vif.overflow <= 1;
        else
            fifo_vif.overflow <= 0;
    end
end
```

```
always @(posedge fifo_vif.clk or negedge fifo_vif.rst_n) begin
    if (!fifo_vif.rst_n) begin
        rd_ptr <= 0;
        fifo_vif.underflow <= 0;  // this was added
        fifo_vif.data_out <= 0; // this was added
    end
    else if (fifo_vif.rd_en && count != 0) begin
        fifo_vif.data_out <= mem[rd_ptr];
        rd_ptr <= rd_ptr + 1;
        fifo_vif.underflow <= 0;  // this was added
    end else begin
        if (fifo_vif.rd_en && fifo_vif.empty) // this synchronized
            fifo_vif.underflow <= 1;
        else
            fifo_vif.underflow <= 0;
    end
end

always @(posedge fifo_vif.clk or negedge fifo_vif.rst_n) begin
    if (!fifo_vif.rst_n) begin
        count <= 0;
    end
    else begin
        if  ( ({fifo_vif.wr_en, fifo_vif.rd_en} == 2'b10) && !fifo_vif.full)
            count <= count + 1;
        else if ( ({fifo_vif.wr_en, fifo_vif.rd_en} == 2'b01) && !fifo_vif.empty)
            count <= count - 1;
        else if({fifo_vif.wr_en, fifo_vif.rd_en} == 2'b11) begin // this condition was added
            if(fifo_vif.empty)
                    count <= count + 1;
            else if(fifo_vif.full)
                    count <= count - 1;
        end
    end
end
```

```
assign fifo_vif.full = (count == fifo_vif.FIFO_DEPTH)? 1 : 0;
assign fifo_vif.empty = (count == 0)? 1 : 0;
// assign fifo_vif.underflow = (fifo_vif.empty && fifo_vif.rd_en)? 1 : 0; // this was synchronized
assign fifo_vif.almostfull = (count == fifo_vif.FIFO_DEPTH-1)? 1 : 0;
assign fifo_vif.almostempty = (count == 1)? 1 : 0;

`ifdef SIM

    property Reset_Behavior;
        @(posedge fifo_vif.clk) (!fifo_vif.rst_n) |=> (count == 0 && wr_ptr == 0 && rd_ptr == 0);
    endproperty

    assert property(Reset_Behavior);
    cover property(Reset_Behavior);

    property Write_Acknowledge;
        @(posedge fifo_vif.clk) disable iff (!fifo_vif.rst_n)
        (fifo_vif.wr_en && !fifo_vif.full) |=> (fifo_vif.wr_ack == 1);
    endproperty

    assert property(Write_Acknowledge);
    cover property(Write_Acknowledge);

    property overflow_detection;
        @(posedge fifo_vif.clk) disable iff (!fifo_vif.rst_n)
        (fifo_vif.wr_en && fifo_vif.full) |=> (fifo_vif.overflow == 1);
    endproperty

    assert property(overflow_detection);
    cover property(overflow_detection);

    property underflow_detection;
        @(posedge fifo_vif.clk) disable iff (!fifo_vif.rst_n)
        (fifo_vif.rd_en && fifo_vif.empty) |=> (fifo_vif.underflow == 1);
    endproperty

```

```
property underflow_detection;
    @(posedge fifo_vif.clk) disable iff (!fifo_vif.rst_n)
    (fifo_vif.rd_en && fifo_vif.empty) |=> (fifo_vif.underflow == 1);
endproperty

assert property(underflow_detection);
cover property(underflow_detection);

property Full_Flag;
    @(posedge fifo_vif.clk) disable iff (!fifo_vif.rst_n)
    (count == fifo_vif.FIFO_DEPTH) |-> (fifo_vif.full == 1);
endproperty
assert property(Full_Flag);
cover  property(Full_Flag);

property Empty_Flag;
    @(posedge fifo_vif.clk) disable iff (!fifo_vif.rst_n)
    (count == 0) |-> (fifo_vif.empty == 1);
endproperty
assert property(Empty_Flag);
cover  property(Empty_Flag);

property AlmostFull_Flag;
    @(posedge fifo_vif.clk) disable iff (!fifo_vif.rst_n)
    (count == fifo_vif.FIFO_DEPTH - 1) |-> (fifo_vif.almostfull == 1);
endproperty
assert property(AlmostFull_Flag);
cover  property(AlmostFull_Flag);
```

```
    property AlmostFull_Flag;
        @(posedge fifo_vif.clk) disable iff (!fifo_vif.rst_n)
        (count == fifo_vif.FIFO_DEPTH - 1) |-> (fifo_vif.almostfull == 1);
    endproperty
    assert property(AlmostFull_Flag);
    cover  property(AlmostFull_Flag);

    property AlmostEmpty_Flag;
        @(posedge fifo_vif.clk) disable iff (!fifo_vif.rst_n)
        (count == 1) |-> (fifo_vif.almostempty == 1);
    endproperty

    assert property(AlmostEmpty_Flag);
    cover  property(AlmostEmpty_Flag);

    property WrPtr_Wraparound;
        @(posedge fifo_vif.clk) disable iff (!fifo_vif.rst_n)
        (wr_ptr == fifo_vif.FIFO_DEPTH - 1 && fifo_vif.wr_en && !fifo_vif.full)
        |=> (wr_ptr == 0);
    endproperty

    assert property(WrPtr_Wraparound);
    cover  property(WrPtr_Wraparound);

    property RdPtr_Wraparound;
        @(posedge fifo_vif.clk) disable iff (!fifo_vif.rst_n)
        (rd_ptr == fifo_vif.FIFO_DEPTH - 1 && fifo_vif.rd_en && !fifo_vif.empty)
        |=> (rd_ptr == 0);
    endproperty
    assert property(RdPtr_Wraparound);
    cover  property(RdPtr_Wraparound);
```

```
    property Ptr_Threshold;
        @(posedge fifo_vif.clk) disable iff (!fifo_vif.rst_n)
        (wr_ptr < fifo_vif.FIFO_DEPTH) &&
        (rd_ptr < fifo_vif.FIFO_DEPTH) &&
        (count  <= fifo_vif.FIFO_DEPTH);
    endproperty
    assert property(Ptr_Threshold);
    cover  property(Ptr_Threshold);

`endif

endmodule
```

FIFO_pkg.sv

```
package shared_pkg;

    static bit test_finished = 0;
    static int error_count = 0;
    static int correct_count = 0;

endpackage
```

FIFO_transaction.sv

```
package FIFO_transaction_pkg;
import shared_pkg::*;

    class FIFO_transaction #(FIFO_WIDTH = 16, FIFO_DEPTH = 8);
        rand logic [FIFO_WIDTH-1:0] data_in;
        rand logic rst_n, wr_en, rd_en;
        logic [FIFO_WIDTH-1:0] data_out;
        logic wr_ack, overflow;
        logic full, empty, almostfull, almostempty, underflow;

        int RD_EN_ON_DIST = 0;
        int WR_EN_ON_DIST = 0;

        function new(int RD_EN_ON_DIST  = 30,int WR_EN_ON_DIST = 70);
            this.RD_EN_ON_DIST = RD_EN_ON_DIST;
            this.WR_EN_ON_DIST = WR_EN_ON_DIST;
        endfunction

        // constraint
        constraint rst_cst{
            rst_n dist{0 := 2, 1 := 98};
        }

        constraint wr_cst{
            wr_en dist{0 := 100 - WR_EN_ON_DIST, 1 := WR_EN_ON_DIST};
        }

        constraint rd_cst{
            rd_en dist{0 := 100 - RD_EN_ON_DIST, 1 := RD_EN_ON_DIST};
        }

    endclass

endpackage
```

FIFO_coverage.sv

```
package FIFO_coverage_pkg;
import FIFO_transaction_pkg::*;
import shared_pkg::*;

    class FIFO_coverage;
        FIFO_transaction F_cvg_txn;

        covergroup FIFO_cvg;

            wr_cvg: coverpoint F_cvg_txn.wr_en;
            rd_cvg: coverpoint F_cvg_txn.rd_en;
            wr_ack_cvg : coverpoint F_cvg_txn.wr_ack;
            overflow_cvg : coverpoint F_cvg_txn.overflow;
            full_cvg : coverpoint F_cvg_txn.full;
            empty_cvg : coverpoint F_cvg_txn.empty;
            almostfull_cvg : coverpoint F_cvg_txn.almostfull;
            almostempt_cvg : coverpoint F_cvg_txn.almostempty;
            underflow_cvg : coverpoint F_cvg_txn.underflow;

            wr_ack_C: cross wr_cvg, rd_cvg , wr_ack_cvg;
            overflow_C: cross wr_cvg, rd_cvg , overflow_cvg;
            full_C: cross wr_cvg, rd_cvg , full_cvg;
            empty_C: cross wr_cvg, rd_cvg , empty_cvg;
            almostfull_C: cross wr_cvg, rd_cvg , almostfull_cvg;
            almostempty_C: cross wr_cvg, rd_cvg , almostempt_cvg;
            underflow_C: cross wr_cvg, rd_cvg , underflow_cvg;

        endgroup

        function new();
            FIFO_cvg = new();
            F_cvg_txn = new();
        endfunction

        function void sample_data(FIFO_transaction F_txn);
            F_cvg_txn = F_txn;
            FIFO_cvg.sample();
        endfunction


    endclass

endpackage
```

```
package FIFO_scoreboard_pkg;
import FIFO_transaction_pkg::*;
import shared_pkg::*;

    class FIFO_scoreboard #(FIFO_WIDTH = 16, FIFO_DEPTH = 8);

        FIFO_transaction F_txn;

        logic [FIFO_WIDTH-1:0] data_out_ref;
        logic wr_ack_ref, overflow_ref;
        logic full_ref, empty_ref, almostfull_ref, almostempty_ref, underflow_ref;

        localparam max_fifo_addr = $clog2(FIFO_DEPTH);
        reg [FIFO_WIDTH-1:0] queue_ref [FIFO_DEPTH-1:0];
        reg [max_fifo_addr-1:0] wr_ptr_ref, rd_ptr_ref;
        reg [max_fifo_addr:0] count_ref;

        task reference_model(FIFO_transaction F_txn);
           // $display("start reference model");
            if(!F_txn.rst_n) begin
                data_out_ref = 0;
                wr_ack_ref = 0;
                overflow_ref = 0;
                underflow_ref = 0;
                wr_ptr_ref = 0;
                rd_ptr_ref = 0;
                count_ref = 0;
                empty_ref = 1;
                full_ref = 0;
                almostfull_ref = 0;
                almostempty_ref = 0;
```

```
            end  else  begin
                // writing
                if(F_txn.wr_en && count_ref < FIFO_DEPTH) begin
                    queue_ref[wr_ptr_ref] = F_txn.data_in;
                    wr_ack_ref = 1;
                    wr_ptr_ref += 1;
                end else begin
                    wr_ack_ref = 0;
                end

                // reading
                if(F_txn.rd_en && count_ref != 0) begin
                    data_out_ref = queue_ref[wr_ptr_ref] ;
                    rd_ptr_ref += 1;
                end

                // sequential flags
                if(count_ref == 0 && F_txn.rd_en)
                    underflow_ref = 1;
                else
                    underflow_ref = 0;

                // sequential flags
                if(count_ref == FIFO_DEPTH  && F_txn.wr_en)
                    overflow_ref = 1;
                else
                    overflow_ref = 0;

```

```
            if  ( ({F_txn.wr_en, F_txn.rd_en} == 2'b10) && !full_ref)
                count_ref <= count_ref + 1;
            else if ( ({F_txn.wr_en, F_txn.rd_en} == 2'b01) && !empty_ref)
                count_ref <= count_ref - 1;
            else if ({F_txn.wr_en, F_txn.rd_en} == 2'b11) begin
                if(empty_ref)
                    count_ref <= count_ref + 1;
                else if(full_ref)
                    count_ref <= count_ref - 1;
            end

            if(count_ref == FIFO_DEPTH) full_ref = 1; else full_ref = 0;
            if(count_ref == 0) empty_ref = 1; else empty_ref = 0;
            if(count_ref == FIFO_DEPTH-1) almostfull_ref = 1; else almostfull_ref = 0;
            if(count_ref == 1) almostempty_ref = 1; else almostempty_ref = 0;

        end

    endtask
```

```
        task check_data(FIFO_transaction F_txn);
            reference_model(F_txn);
            if(data_out_ref !== F_txn.data_out ||
               wr_ack_ref !== F_txn.wr_ack ||
               full_ref != F_txn.full ||
               overflow_ref !== F_txn.overflow ||
               empty_ref !== F_txn.empty ||
               almostfull_ref !== F_txn.almostfull ||
               almostempty_ref !== F_txn.almostempty ||
               underflow_ref !== F_txn.underflow) begin
                error_count++;

            end else begin
                    correct_count++;

            end

        endtask

    endclass

endpackage
```

```
import FIFO_transaction_pkg::*;
import FIFO_coverage_pkg::*;
import FIFO_scoreboard_pkg::*;
import shared_pkg::*;

module FIFO_monitor(FIFO_if.MONITOR fifo_vif);
    FIFO_transaction F_txn;
    FIFO_coverage f_cov;
    FIFO_scoreboard f_scb;

    initial begin
        F_txn = new();
        f_cov = new();
        f_scb = new();

        forever begin
            @(fifo_vif.drv_done);

            F_txn.data_in     = fifo_vif.data_in;
            F_txn.wr_en       = fifo_vif.wr_en;
            F_txn.rd_en       = fifo_vif.rd_en;
            F_txn.rst_n       = fifo_vif.rst_n;

            F_txn.data_out    = fifo_vif.data_out;
            F_txn.full        = fifo_vif.full;
            F_txn.almostfull  = fifo_vif.almostfull;
            F_txn.empty       = fifo_vif.empty;
            F_txn.almostempty = fifo_vif.almostempty;
            F_txn.overflow    = fifo_vif.overflow;
            F_txn.underflow   = fifo_vif.underflow;
            F_txn.wr_ack      = fifo_vif.wr_ack;

            fork
                f_cov.sample_data(F_txn);
                f_scb.check_data(F_txn);
            join

            if(test_finished) begin
                $display("%t: Simulation Finished", $time);
                $display("  Correct: %0d  |  Errors: %0d", correct_count, error_count);
                $stop;
            end
        end
    end

endmodule
```

```
import FIFO_transaction_pkg::*;
import shared_pkg::*;

module FIFO_tb(FIFO_if.TEST fifo_vif);
    FIFO_transaction F_txn = new();

    initial begin
        #1;
        repeat(2) begin
            fifo_vif.rst_n   = 0;
            fifo_vif.wr_en   = 0;
            fifo_vif.rd_en   = 0;
            fifo_vif.data_in = 0;
            @(negedge fifo_vif.clk);
            -> fifo_vif.drv_done;
        end

        fifo_vif.rst_n = 1;
        @(negedge fifo_vif.clk);
        -> fifo_vif.drv_done;

        // to check full flag
        repeat(10) begin
            fifo_vif.wr_en   = 1;
            fifo_vif.rd_en   = 0;
            fifo_vif.data_in = $urandom_range(0, 255);
            @(negedge fifo_vif.clk);
            -> fifo_vif.drv_done;
        end

        fifo_vif.wr_en   = 1;
        @(negedge fifo_vif.clk);
        -> fifo_vif.drv_done;
        fifo_vif.wr_en   = 0;
        @(negedge fifo_vif.clk);
        -> fifo_vif.drv_done;
```

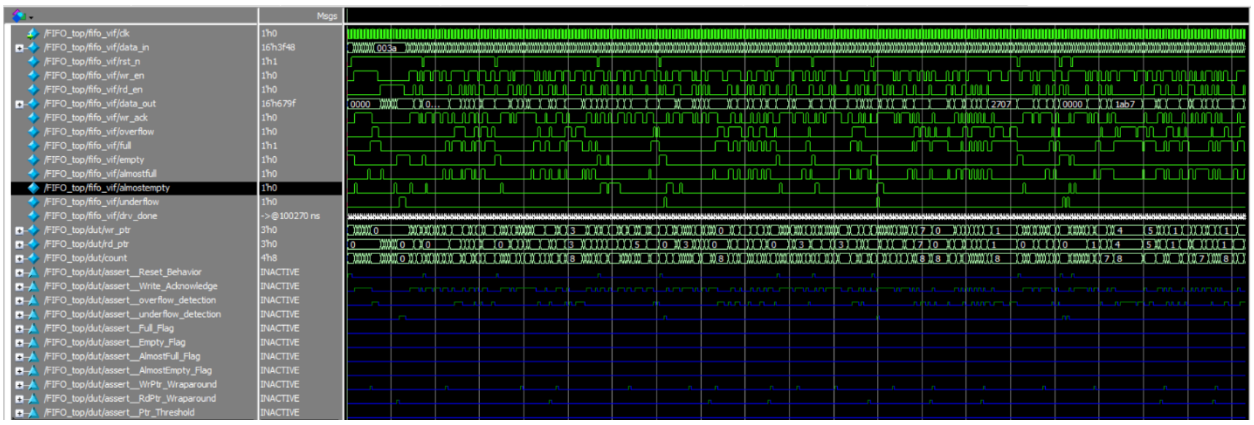
```
        // to check empty flag
        repeat(10) begin
            fifo_vif.rd_en   = 1;
            fifo_vif.wr_en = 0;
            @(negedge fifo_vif.clk);
            -> fifo_vif.drv_done;
        end

        fifo_vif.rd_en   = 1;
        @(negedge fifo_vif.clk);
        -> fifo_vif.drv_done;
        fifo_vif.rd_en   = 0;
        @(negedge fifo_vif.clk);
        -> fifo_vif.drv_done;

        repeat(10000) begin
            assert(F_txn.randomize());
            fifo_vif.rst_n   = F_txn.rst_n;
            fifo_vif.wr_en   = F_txn.wr_en;
            fifo_vif.rd_en   = F_txn.rd_en;
            fifo_vif.data_in = F_txn.data_in;
            @(negedge fifo_vif.clk);
            -> fifo_vif.drv_done;
        end

        test_finished = 1;
    end

endmodule
```

Name	Assertion Type	Language	Enable	Failure Count	Pass Count	Active Count	Memory	Peak Memory	Peak Memory Time	Cumulative Threads	ATV
/FIFO_top/dut/assert__Reset_Behavior	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off
/FIFO_top/dut/assert__Write_Acknowledge	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off
/FIFO_top/dut/assert__overflow_detection	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off
/FIFO_top/dut/assert__underflow_detection	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off
/FIFO_top/dut/assert__Full_Flag	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off
/FIFO_top/dut/assert__Empty_Flag	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off
/FIFO_top/dut/assert__AlmostFull_Flag	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off
/FIFO_top/dut/assert__AlmostEmpty_Flag	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off
/FIFO_top/dut/assert__WrPtr_Wraparound	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off
/FIFO_top/dut/assert__RdPtr_Wraparound	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off
/FIFO_top/dut/assert__Ptr_Threshold	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off
/FIFO_top/test/#ublk#182146786#53/immed__54	Immediate	SVA	on	0	1	-	-	-	-		off

Cover Directives

Name	Language	Enabled	Log	Count	AtLeast	Limit	Weight	Cmplt %	Cmplt graph	Included
/FIFO_top/dut/cover__Ptr_Threshold	SVA	✓	Off	9816	1	Unlimited	1	100%		✓
/FIFO_top/dut/cover__RdPtr_Wraparound	SVA	✓	Off	265	1	Unlimited	1	100%		✓
/FIFO_top/dut/cover__WrPtr_Wraparound	SVA	✓	Off	411	1	Unlimited	1	100%		✓
/FIFO_top/dut/cover__AlmostEmpty_Flag	SVA	✓	Off	428	1	Unlimited	1	100%		✓
/FIFO_top/dut/cover__AlmostFull_Flag	SVA	✓	Off	2602	1	Unlimited	1	100%		✓
/FIFO_top/dut/cover__Empty_Flag	SVA	✓	Off	363	1	Unlimited	1	100%		✓
/FIFO_top/dut/cover__Full_Flag	SVA	✓	Off	4128	1	Unlimited	1	100%		✓
/FIFO_top/dut/cover__underflow_detection	SVA	✓	Off	93	1	Unlimited	1	100%		✓
/FIFO_top/dut/cover__overflow_detection	SVA	✓	Off	2799	1	Unlimited	1	100%		✓
/FIFO_top/dut/cover__Write_Acknowledge	SVA	✓	Off	3939	1	Unlimited	1	100%		✓
/FIFO_top/dut/cover__Reset_Behavior	SVA	✓	Off	211	1	Unlimited	1	100%		✓

Name	Class Type	Coverage	Goal	% of Goal	Status	Included
/FIFO_coverage_pkg/FIFO_coverage		100.00%				
TYPE FIFO_cvg		100.00%	100	100.00...		✓
CVP FIFO_cvg::wr_cvg		100.00%	100	100.00...		✓
CVP FIFO_cvg::rd_cvg		100.00%	100	100.00...		✓
CVP FIFO_cvg::wr_ack_cvg		100.00%	100	100.00...		✓
CVP FIFO_cvg::overflow_cvg		100.00%	100	100.00...		✓
CVP FIFO_cvg::full_cvg		100.00%	100	100.00...		✓
CVP FIFO_cvg::empty_cvg		100.00%	100	100.00...		✓
CVP FIFO_cvg::almostfull_cvg		100.00%	100	100.00...		✓
CVP FIFO_cvg::almostempt_cvg		100.00%	100	100.00...		✓
CVP FIFO_cvg::underflow_cvg		100.00%	100	100.00...		✓
CROSS FIFO_cvg::wr_ack_C		100.00%	100	100.00...		✓
CROSS FIFO_cvg::overflow_C		100.00%	100	100.00...		✓
CROSS FIFO_cvg::full_C		100.00%	100	100.00...		✓
CROSS FIFO_cvg::empty_C		100.00%	100	100.00...		✓
CROSS FIFO_cvg::almostfull_C		100.00%	100	100.00...		✓
CROSS FIFO_cvg::almostempty_C		100.00%	100	100.00...		✓
CROSS FIFO_cvg::underflow_C		100.00%	100	100.00...		✓



Statements - by instance (/FIFO_top/dut)

FIFO.sv

```
17 always @(posedge fifo_vif.clk or negedge fifo_vif.rst_n) begin
19 wr_ptr <= 0;
20 fifo_vif.wr_ack <= 0; // this was added
21 fifo_vif.overflow <= 0; // this was added
24 mem[wr_ptr] <= fifo_vif.data_in;
25 fifo_vif.wr_ack <= 1;
26 wr_ptr <= wr_ptr + 1;
29 fifo_vif.wr_ack <= 0;
31 fifo_vif.overflow <= 1;
33 fifo_vif.overflow <= 0;
37 always @(posedge fifo_vif.clk or negedge fifo_vif.rst_n) begin
39 rd_ptr <= 0;
40 fifo_vif.underflow <= 0;  // this was added
41 fifo_vif.data_out <= 0; // this was added
44 fifo_vif.data_out <= mem[rd_ptr];
45 rd_ptr <= rd_ptr + 1;
46 fifo_vif.underflow <= 0;  // this was added
49 fifo_vif.underflow <= 1;
51 fifo_vif.underflow <= 0;
55 always @(posedge fifo_vif.clk or negedge fifo_vif.rst_n) begin
57 count <= 0;
61 count <= count + 1;
63 count <= count - 1;
66 count <= count + 1;
68 count <= count - 1;
73 assign fifo_vif.full = (count == fifo_vif.FIFO_DEPTH)? 1 : 0;
74 assign fifo_vif.empty = (count == 0)? 1 : 0;
76 assign fifo_vif.almostfull = (count == fifo_vif.FIFO_DEPTH-1)? 1 : 0;
77 assign fifo_vif.almostempty = (count == 1)? 1 : 0;
```

FIFO.sv

```
18 if (!fifo_vif.rst_n) begin
23 else if (fifo_vif.wr_en && count < fifo_vif.FIFO_DEPTH) begin
28 else begin
30 if (fifo_vif.full & fifo_vif.wr_en)
32 else
38 if (!fifo_vif.rst_n) begin
43 else if (fifo_vif.rd_en && count != 0) begin
47 end else begin
48 if (fifo_vif.rd_en && fifo_vif.empty) // this synchronized
50 else
56 if (!fifo_vif.rst_n) begin
59 else begin
60 if       ( ({fifo_vif.wr_en, fifo_vif.rd_en} == 2'b10) && !fifo_vif.full)
62 else if ( ({fifo_vif.wr_en, fifo_vif.rd_en} == 2'b01) && !fifo_vif.empty)
64 else if({fifo_vif.wr_en, fifo_vif.rd_en} == 2'b11) begin // this condition was added
65 if(fifo_vif.empty)
67 else if(fifo_vif.full)
73 assign fifo_vif.full = (count == fifo_vif.FIFO_DEPTH)? 1 : 0;
74 assign fifo_vif.empty = (count == 0)? 1 : 0;
76 assign fifo_vif.almostfull = (count == fifo_vif.FIFO_DEPTH-1)? 1 : 0;
77 assign fifo_vif.almostempty = (count == 1)? 1 : 0;
```

Toggles - by instance (/FIFO_top/fifo_vif)

- sim:/FIFO_top/fifo_vif
  - almostempty
  - almostfull
  - clk
  - data_in
  - data_out
  - empty
  - full
  - overflow
  - rd_en
  - rst_n
  - underflow
  - wr_ack
  - wr_en